

复数部分:

`d = complex(6, 9)` 得到 `6+9i` `complex` 是复数类型

`e = real(d)` 得到 `6` %取实数部分

`f = imag(d)` 得到 `9` %取虚部

`g = abs(d)` %求复数的模

求辐角: `angle(d)`

取共轭: `conj(d)`

查看变量 `d` 的类型: `class(d)`

矩阵:

矩阵可拼接, 如: `A = [1 2 3;4 5 6];B = [5 6 8;9 0 4];C = [A B;B A]`

随机生成一个 4×4 矩阵, 其中每个元素都在 `0,1` 之间: `A = rand(4,4)`

生成一个行向量: `a = 1:2:19` `start : walk : end`

或: `b = linspace(1, 10, 20)` 从 `1` 到 `10` 等间隔分为 `20` 个

矩阵索引: 下标: 如 `A(2, 3)`是矩阵 `A` 第二行第三列元素

序号: 如 `A(7)`是指矩阵中数到的第七个元素(按列来数, 从上到下, 左到右)

求矩阵大小: `length(矩阵)` %取出矩阵最长的维度的长度。如放入一个 4×6 矩阵得到 `6`

求矩阵维数: `ndims(矩阵)`

求矩阵内元素数量: `numel(矩阵)`

`A(3,:)`表示输出矩阵 `A` 第三行全部元素, 后面 `“:”` 可以看作 `all`。同理, `A(:,2)`输出第二列全部元素。`A(1:3,2:4)`取出既在第一到第三行, 又在第二到第四列的所有元素。`A(:,1:3)`输出 `1` 到 `3` 列所有元素。`A(:,1:3:5)`取出第一第四列所有元素(`start:walk:end`)

`reshape(矩阵,x,y)` 把输入的矩阵变成 `x` 行 `y` 列

常用内部函数:

`sqrt(数字)`对数字开根号

`exp(x)`是 `e` 的 `x` 次方

`abs(数字)`取数字的绝对值

`cosd(47)`是 `cos47°`

`cos(47)`是弧度为 `47` 的角的余弦

`log(10)`表示 `ln10`

`log10(100)`表示以 `10` 为底 `100` 的对数

`pow2(16)`表示 `2` 的 `16` 次方

`gcd(12, 82)`求 `12` 和 `82` 的最大公约数

`factorial(5)`求 `5` 的阶乘

`num2str(变量)`把变量变成字符串

`double(变量)`把结果变成双精度数值

`mod(a,b)`得到 `a` 除以 `b` 的余数

`x = input('输入一个数字: ')` 若输入多个数字要用逗号分隔, 最后存储在数组内

`Name = input('输入姓名: ','s')` `'s'`表示将输入数据转换为字符串类型

算数, 关系和逻辑:

`.*` `A.*B` 是矩阵 `A` 和矩阵 `B` 中元素对应相乘

`*` `A*B` 是矩阵乘矩阵

点除，除与之类似

关系运算中，==是等于，~=是不等于

逻辑运算中，&是“与”，|是“或”，~是“非”

字符串：

str1 = 'MATLAB'可看作一个行向量 str1(2)是'A'

str2 = 'Nikepp'

strA = [str1;str2]把两个字符串放在一个矩阵里。对 strA 的操作与对矩阵一致

稀疏矩阵：

```
A1 = [2 0 0 0;  
      0 0 0 0;  
      0 -5 0 0;  
      3 0 0 0]  
% 普通方式存储 2 0 0 3 0 0 -5 0 0 0 0 0 0 0 0  
% 稀疏方式存储 (1,1),2,(4,1),3,(3,2),-5
```

稀疏方式存储时格式为 下标+元素

S2 = sparse(4,6)产生一个 4×6 的全 0 稀疏矩阵

S = [-5 24 9 17] u = [1 3 5 2] v = [1 2 2 4]

S3 = sparse(u, v, S) full(S3)

% 产生一个稀疏矩阵，u 为行标，v 为列标，s 为非零元素数字，如(2,4)处存储元素为 17

[u, v, S] = find(A) full(S3)返回矩阵 A 中非零元素的行标列标和元素，分别由 u,v,S 接收

便捷创建一个稀疏存储矩阵：

A2 = [2 2 1;3 1 -1;4 3 3] S4 = spconvert(A2) 得到 S4，其中(2,2)为 1, (3,1)为-1，以此类推

full(S4) 将之充满变成一个稀疏矩阵

创建对角稀疏矩阵：

```
% 创建对角稀疏矩阵 spdiags() 梳状稀疏矩阵  
B = [0 1 4 9 0;  
      4 7 9 0 3;  
      0 2 6 4 9]  
Bz = B.'  
d = [-1,0,1].'  
full(spdiags(Bz,d,5,5))
```

B 是三条对角线的元素，d 是对角线位置，如-1 对角线代表 0 1 4 9 0。-1 对角线只有前几个元素被作为存储元素，而+1 对角线是后几个元素作为存储元素。因为负对角线和主对角线长度不一致，部分元素被挤掉。

因为存储元素是每一列而不是每一行，要对 B 和 d 进行转置

spdiags()函数内 5,5 表示产生一个 5×5 方阵

稀疏矩阵应用举例

求解下列三对角线性方程组的解

求下列三对角线性方程组的解。

$$\begin{bmatrix} 2 & 3 & & & \\ 1 & 4 & 1 & & \\ & 1 & 6 & 4 & \\ & & 2 & 6 & 2 \\ & & & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \\ 2 \\ 1 \\ 5 \end{bmatrix}$$

分析：左侧系数矩阵是一个稀疏矩阵，只有三条对角线上有元素

```
% tips:当参与运算的矩阵不全是稀疏矩阵时，运算结果一般为完全存储方式
% ex2:
C1 = [1 1 2 1 0;2 4 6 6 1;0 3 1 4 2].';
d1 = [-1 0 1].';

[C] = full(spdia(C1,d1,5,5))
V2 = [0 3 2 1 5].';

% Ax = b
% 即 x = A^-1*b
x = (inv(C)*V2).';
```

设置一个函数,如:

function [V] = func1(r,h) %function 不能少

V = pi*r*r*h %函数体

end %函数结束

函数文件声明了一个函数，方便调用

数据输入：input() 如：a = input("请输入数值 a:")

数据输出：disp(a) %类似 python 的 print

程序暂停：pause(5) %暂停 5 秒

if-elseif-else 语句后面没有冒号

最后加 end

if 是顺序执行，判断完第一个再判断下一个条件。switch 是并行执行，会同时下方判断所有条件，进入第一个为 true 的 case。注意 switch 与 case 和 otherwise 搭配使用

rand 函数:

直接输入 rand 并回车，生成一个范围[0, 1]的随机数，可进行加减乘除运算

也可作为函数:

rand(3)生成一个三阶方阵，其中每个元素都是[0, 1]的随机数

rand(1,2)生成一个一行二列矩阵

向上取整：ceil(变量)

向下取整：floor(变量)

向零取整（正数就向下取整，负数向上取整）：fix(变量)

四舍五入：round(变量)

roundn(ans,-2): 对 ans 保留两位小数（四舍五入）

for i=1:100 表示循环 100 次，i 从 1 到 100

用 for 循环赋值，系统默认不显示，后面不用加分号。循环结束后写 end

while 语句自定义退出条件，用于不知道要循环几次时

变量 x 无具体值时，称 x 为符号变量

定义符号变量 x : `syms x`

用 x 构造函数: `y = x*3`

求 y 的导数并赋值给 $y1$: `y1=diff(y)`

`diff(y,n)`对函数 y 求导 n 次 (用于一元函数)

`diff(y,'t',n)`对函数 y 的自变量 t 求 n 次偏导 (用于多元函数)

`subs(y1,x,1)` 对 x 赋值 1 求 $y1$

求方程的解:

①`roots()`函数用于求解多项式的根。它的用法是给定多项式的系数向量作为输入,并返回多项式的根。每个找到的根都以列向量的形式返回。

如: 对于方程 $x^3 - x = 0$, 其系数向量是 `[1, 0, -1]`, 求根:

```
p = [1, 0, -1];
```

```
r = roots(p);
```

最后返回一个根向量 `r`, 其中包含方程 $x^3 - x = 0$ 的解。

②`fzero()`函数:如: 对于方程 $xe^x = e$, 可以将其转化为 $f(x) = xe^x - e = 0$ 的形式。

```
f = @(x) x*exp(x) - exp(1);
```

```
x0 = 1; % 初始猜测值
```

```
x = fzero(f, x0);
```

首先, 定义匿名函数 $f(x) = xe^x - e$ 。然后, 指定一个接近方程解的初始猜测值 $x0$ 。

最后, 使用 `fzero()`函数求解方程的根。函数将返回方程的一个根, 存储在变量 `x` 中。`fzero()`函数在给定初始猜测值的情况下, 可能只找到一个根, 解决方法之一是提供不同的初始猜测值并对解进行验证。

`break` 跳出距离它最近的循环语句, 多层嵌套时跳出其身处的最里层循环

写完代码该行末尾加分号会执行计算, 但不在命令行窗口显示输出

命令行输入 `clc`(清空命令行), 输入 `clear`(清空工作区), 按上方向键调用历史指令

插入分节符, 点运行节, 只运行运行节内代码

字符串用双引号括起来, 字符串内有双引号则在内部的双引号位置用双重双引号

字符串添加内容: `"dhdhdb" + "dhdh"`

一次存多个元素 (矩阵):

```
A=["a","ccc";"dddd","eeee"]运行后得到矩阵
```

`"abcde"`: 1×1 string (字符串) (一个变量)

`'abcde'`: 1×5 char (字符) (多个变量)

字符串本身有单引号, 在单引号位置用双重单引号

`DNA='abcde'` `D= "ajsbsv"`

DNA(4)是'd' (跟 py 不一样)
D(4)不存在，因为是 1×1 string

创建 1×6 矩阵：
a=[1 2 3 4 5 6] （没有逗号）
对矩阵中每个元素+3：
b = a + 3

plot 函数作图，以索引为横坐标。第一个索引是 1 不是 0
a=[1 2 3 4 5 6]
plot(a) %作图
grid on %添加网格线

多维矩阵：以空格或逗号分隔同一行元素，分号分隔各行
A=[3 2 1;2 4 1;3 9 2]
B=A' %求转置矩阵
[D,V]=eig(A) %求特征值 D 和特征向量 V
E=inv(A) %求逆矩阵

矩阵乘法和矩阵点乘：
C=A*B %矩阵乘法
C=A.*B %点乘，即对应元素相乘

A 的逆矩阵乘矩阵 b（求解 $A*x=b$ ）：
x=A \ b %A 在反斜杠相对下方表示 A 的逆矩阵

一个操作数是标量另一个不是，自动把该标量隐式拓展为与另一操作数相同大小：
H=[1 1 1;2 2 2;3 3 3]
K=4
L=K*H %运算时 K 变成 3×3 对角矩阵
M=K+H %运算时 K 变成 3×3 的矩阵

不同维度行向量和列向量相加：
补全维度。一般不要进行。

二．绘制四种二维图：

（1）线图

x = 0:0.05:30; %从 0 到 30，每隔 0.05 取值一次
y = sin(x);
plot(x,y) %若改成 (x,y, 'LineWidth',2)可把线条变粗
xlabel("横轴标题") %设置横轴纵轴标题
ylabel("纵轴标题")

```
grid on %显示网格
axis(0 20 -1.5 1.5) %设置横纵坐标范围
```

需要多组函数显示在同一张图：

```
y1 = sin(x);
y2 = cos(x);
plot(x,y1,x,y2)
axis(0 20 -1.5 1.5)
```

（2）条形图

bar 函数创建垂直条形图，barh 函数创建水平条形图

```
t = -3:0.5:3;
p = exp(-t.*t); %点乘，表示 e 的 -t2
bar(t,p)
barh(t,p)
```

(3) 极坐标图

polarplot 函数用来绘制极坐标图

```
theta = 0:0.01:2*pi; %弧度。pi 表示圆周率，*不能省略
radi = abs(sin(2*theta).*cos(2*theta)); %半径。abs 求绝对值或复数的模
polarplot(theta,radi) %括号内是弧度和半径
```

（4）散点图

scatter 函数绘制 x 和 y 值的散点图，如：

```
Height = randn(1000,1)
%随机生成一个 1000×1 的矩阵，即 1000 个随机数，这些数符合正态分布
Weight = randn(1000,1)
scatter(Height,Weight)
xlabel('Height')
ylabel('Weight')
```

三．绘制三维图和内嵌子图

（1）三维曲面图

用 surf 函数。一般是展示 $z = z(x,y)$ 的图像，如：

```
[x,y] = meshgrid(-2:0.2:2);
```

```
%用 meshgrid 创建空间上 (x,y)点。-2 到 2 每隔 0.2 取一点
z = x.*exp(-x.^2+y.^2);
surf(x,y,z); #如果 x,y,z 是离散数据, 输入时 x,y 的行列与 z 一致
colormap hsv %设置颜色。hsv 常用, winter 冷色调, summer 暖色调
colorbar %图右边出现一个条状图, 展示与数值对应的颜色色调
```

(2) 子图

用 subplot 函数可以在同一窗口不同子区域显示多个绘图, 如:

```
theta = 0:0.01:2*pi;
radi = abs(sin(2*theta).*cos(2*theta));
Height = randn(1000,1);
Weight = randn(1000,1);
subplot(2,2,1);surf(x.^2);title('1st'); %(2,2,1)表示放两行两列四个图, 该图第一
subplot(2,2,2);surf(y.^3);title('2nd'); %surf 表示绘制三维曲面图
subplot(2,2,3);polarplot(theta,radi);title('3rd');
subplot(2,2,4);scatter(Height,Weight);title('4th'); %title 设置标题可随意命名
```

四. 导入数据

只能识别英文

Ctrl+s 选择 excel 表格中要导入的部分

常用列向量 (每一列导入为单个 $m \times 1$ 向量), 数值矩阵

五. 处理缺失值和异常值

①清理缺失数:

先写一组含缺失值和异常值的例子:

```
X = 1:100; %构造一数组, 元组为 1, 2, 3……100
Data = randn(1,100); %生成一个  $1 \times 100$  矩阵, 其元素均值为 0, 方差为 1, 符合正态分布
Data(20:20:80) = NaN; %从 20 到 80, 每隔 20 个, 即第 20, 40, 60, 80 个元素设为缺失值
Data(10) = -50; %设置异常值
Data(40) = 45;
Data(70) = -40;
plot(X, Data)
```

实时编辑器--任务--清理缺失数据--设置输入数据--设置清理方法 (填充缺失, **插值)

②清理异常值:

实时编辑器--任务--清理离群数据--设置输入数据--设置清理方法(填充离群值, **插值)
'定义离群值'中的检测方法常用均值、中位数

六.

①线性规划:

□Linprog函数

- 模型化为matlab标准型：目标函数最小值、约束条件小于等于号或等号
- 如果求最大值怎么办？约束条件有大于等于怎么办？

f	目标函数的系数列向量
A, b	不等式约束条件的变量系数矩阵和常数项矩阵
Aeq, beq	等式约束条件的系数矩阵和常数项矩阵
lb, ub	决策变量的最小取值和最大取值

- $[x, fval] = \text{linprog}(f, A, b, Aeq, beq, lb, ub)$
- x 返回最优解的变量取值, $fval$ 返回目标函数的最优值
- 若不存在不等式约束, 用“[]”代替 A 和 b : $[x, fval] = \text{linprog}(f, [], [], Aeq, beq, lb, ub)$
- 若不存在等式约束, 用“[]”代替 Aeq 和 beq : $[x, fval] = \text{linprog}(f, A, b, [], [], lb, ub)$
- 没有等式约束和最小、最大取值的约束时, 可以不写 Aeq, beq 和 lb, ub : $[x, fval] = \text{linprog}(f, A, b)$
- 若题目求最大值: 目标函数等号两端加符号转为求最小值; 求解后目标值再取负

$$\begin{aligned} \max y &= 40x_1 + 30x_2 \\ \text{s.t.} \quad &\begin{cases} x_1 + x_2 \leq 6 \\ x_1 \geq 1, x_2 \geq 1 \\ 240x_1 + 120x_2 \leq 1200 \end{cases} \end{aligned}$$

$$\begin{aligned} \min w &= \begin{bmatrix} -40 \\ -30 \end{bmatrix}^T \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \\ \text{s.t.} \quad &\begin{cases} \begin{bmatrix} 1 & 1 \\ -1 & 0 \\ 0 & -1 \\ 240 & 120 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \leq \begin{bmatrix} 6 \\ -1 \\ -1 \\ 1200 \end{bmatrix} \\ [1, 1] \cdot [x_1, x_2]^T \geq [0, 0] \end{cases} \end{aligned}$$

求 y 最大值, 即求 $-y$ 最小值。计算中只能用 $\leq$, 需要 $\geq$ 时利用负号转化

```
f = [-40;-30]; %目标函数中变量的系数矩阵
a = [1,1;-1,0;0,-1;240,120]; %小于等于的约束条
b = [6;-1;-1;1200]; %小于等于的约束条

%标准形式: [x, fval] = linprog(f,A,b,aeq,beq,lb,
%从某个变量开始后后面都没有? 全都不写
%没有等式约束条件、但有上下界约束时, 用 []在函数里占
[x,y] = linprog(f,a,b)
y = -y

% zeros(m,n)是元素全为0的m行n列的矩阵
```

```
Optimal solution found.
x = 2x1
    4.0000
    2.0000
y = -220
y = 220
```

②非线性规划:

非线性规划-简介

数字建模I

□有限的条件下，最大的收益

- 线性规划：所有变量都是一次方
- 非线性规划：**至少一个变量不是一次方**

• 例： $\min f(x) = x_1^2 + x_2^2 + x_3^2 + 8$

$$\begin{cases} x_1^2 - x_2 + x_3^2 \geq 0 \\ x_1 + x_2^2 + x_3^3 \leq 20 \\ -x_1 - x_2^2 + 2 = 0 \\ x_2 + 2x_3^2 = 3 \\ x_1, x_2, x_3 \geq 0 \end{cases}$$

fun	单独脚本文件里定义的目标函数
x0	决策变量的初始值。不知道的话随便写个数
A, b	线性约束的不等式变量系数矩阵和常数项矩阵(都是 $\leq$)
Aeq, beq	线性约束的等式变量系数矩阵和常数项矩阵
lb, ub	决策变量的最小取值和最大取值
nonlcon	非线性约束，包括不等式和等式

• 函数求解：

- `[x,fval] = fmincon( fun, x0, A, b, Aeq, beq, lb, ub, nonlcon)`
- x得到决策变量取值，fval得到最优解取值
- 本题：设好后fun和nonlcon函数文件以及其他参数后，执行：
- `[x,y] = fmincon('fun1', [0;0;0], [], [], [], [], [0;0;0], [], 'fun2' )`

上面表格中元素没有约束时写[]。

最下面'fun1'和'fun2'对应表中 fun 和 nonlcon，需要提前设定函数文件，单引号表示调用函数。

这里的 fun1:

`function f = fun1(x) %定义函数`

`f = sum(x.^2)+8; %sum 表示求和，即求给出的 x(1)到 x(3)的和`

这里的 fun2:

`function [g,h] = fun2(x) %函数最终返回矩阵[g,h]`

`g = [-x(1)^2+x(2)-x(3)^2 %g 是非线性不等式.要变成 $\leq$ 故取负号
x(1)+x(2)^2+x(3)^3-20]`

`h = [-x(1)-x(2)^2+2 %h 是非线性等式`

`x(2)+2*x(3)^2-3]`

③多目标规划:

需要达到多个目标，前面例题只追求利润一个目标。

适用赛题：使***最多/少/利润最大的同时，尽量……

实现“充分利用”目标也就是尽量“=”

□ 需要衡量每个目标的**完成情况**，并主观上区分三个目标的**重要性**，使得**整体的完成情况**尽量好

□ 引入三个概念：正负偏差变量，绝对约束和目标约束，优先因子

□ 1、衡量每个目标的**完成情况**：正负偏差变量

- 设 $f_i (i = 1, \dots, l)$ 为第 i 个目标函数，
- 正偏差变量 $d_i^+ = \max\{f_i - d_i^0, 0\}$ 表示实际值**超过**目标值的部分，负偏差变量 $d_i^- = \max\{d_i^0 - f_i, 0\}$ 表示实际值**未达到**目标值的部分，这里 d_i^0 表示 f_i 的目标值。
- 例如本题，目标3：尽可能使利润不少于56万，那么 $d_3^0 = 56$ ，

目标函数实际值 f_3	目标值 d_3^0	正偏差变量 d_3^+	负偏差变量 d_3^-	意义
假设利润50万	不少于56万	0	6	离目标差6万
假设利润60万		4	0	超出目标4万

□ 该目标是“尽可能使利润不少于56万”，超了好还是差了好？当然超了好！而且超再多也不怕。

□ 尽量不少于，意味着负偏差变量 d_3^- 越小越好。

正负偏差变量中至少有一个是 0

□ 引入三个概念：正负偏差变量，绝对约束和目标约束，优先因子

□ 绝对约束和目标约束

- 绝对约束是模型中自带的约束条件，必须满足，否则是不可行解

- 例如 $2x_1 + x_2 \leq 11$ 使用材料的数量不能超过总共数量

□ 目标约束是模型中对不等式右端追求的值允许有偏差

- 目标3：尽可能使利润不少于56万，也就是 $z = 8x_1 + 10x_2 \geq 56$ ，但允许有偏差

- 这个“偏差”就是加入正负偏差变量，变成： $8x_1 + 10x_2 + d_3^- - d_3^+ = 56$

□ 利用了允许的“偏差”即正负偏差变量，多退少补，把目标函数变成了等式约束条件

尽可能不少于 56 万，实际上可以少于 56，依照情况

多退少补， $\leq$ 号时加负偏差变量减正偏差变量， $\geq$ 时相反

□ 需要衡量每个目标的**完成情况**，并主观上区分三个目标的**重要性**，使得**整体的完成情况**尽量好

□ 引入三个概念：正负偏差变量，绝对约束和目标约束，优先因子

□ 优先因子

- 1. 尽量使产品I的产量不超过产品II的产量；2. 尽可能充分利用所有设备；3. 尽可能使利润不少于56万
- 这三者到底哪个重要？需要主观确定优先因子 P_k 。
- 例如，我认为目标3最重要，给他优先因子是10；目标2第二重要，给他优先因子5；目标1第三重要，给它优先因子是1。那么三个目标重要性之比是10:5:1

□ 第一个目标要求不超过目标值，意味着正偏差变量 d_1^+ 越小越好

目标	意义	目标函数
1. 不超过目标值	正偏差变量 d_1^+ 越小越好	$\min P_1 d_1^+$
2. 恰好达到目标值	正负偏差都尽量小	$\min P_2 (d_2^- + d_2^+)$
3. 不少于目标值	负偏差变量 d_3^- 越小越好	$\min P_3 d_3^-$

优先因子一般通过查阅文献获得。实在查不到时：经查阅资料可得，优先因子……

□既要XXX，又要XXX

□ 某工厂生产产品I和产品II，有关数据如下，有3个目标：1.尽量使产品I的产量不超过产品II的产量；2.尽可能充分利用所有设备；3.尽可能使利润不少于56万

□ 需要衡量每个目标的完成情况，并主观上区分三个目标的重要性，使得整体的完成情况尽量好。

- 把所有的目标约束都加上 $d_i^- - d_i^+$ 变成等式（注意，绝对约束也可以这样做变成等式，视情况而定）
- 按照需要，主观确定确定优先因子 P_k 。

□ 得到多目标规划模型：

$$\begin{aligned} \min & (P_1 d_1^+ + P_2 (d_2^- + d_2^+) + P_3 d_3^-) \\ \text{s. t. } & \begin{cases} 2x_1 + x_2 \leq 11, \\ x_1 - x_2 + d_1^- - d_1^+ = 0, \\ x_1 + 2x_2 + d_2^- - d_2^+ = 10, \\ 8x_1 + 10x_2 + d_3^- - d_3^+ = 56, \\ x_1, x_2, d_i^-, d_i^+ \geq 0, i = 1, 2, 3 \end{cases} \end{aligned}$$

求解方法：搜索MATLAB的fgoalattain函数；或序贯算法；或用Lingo求解

fgoalattain 函数：https://blog.csdn.net/Power1_Power2/article/details/82957815

工程应用中 fgoalattain 函数调用格式如下：

$[x, fval] = \text{fgoalattain}(\text{fun}, x0, \text{goal}, \text{weight}, A, b, Aeq, beq, lb, ub, \text{nonlcon})$

$x0$ 表示初值；

fun 表示要优化的目标函数；

goal 表示函数 fun 要逼近的目标值，是一个向量，它的维数大小等于目标函数 fun 返回向量 F 的维数大小；

weight 表示给定的权值向量，用于控制目标逼近过程的步长；

例 1. 程序（利用 fgoalattain 函数求解）

$$\begin{aligned} \min & (x_1 - 1)^2 + (x_2 - 2)^2 + (x_3 - 3)^2 \\ \min & x_1^2 + 2x_2^2 + 3x_3^2 \\ \text{s.t. } & x_1 + x_2 + x_3 = 6 \\ & x_1, x_2, x_3 \geq 0 \end{aligned}$$

①建立 M 文件.

`function f=myfun(x)`

`f(1)= x(1)-1)^2+(x(2)-2)^2+(x(3)-3)^2;`

`f(2)= x(1)^2+2*x(2)^2+3*x(3)^2;`

②在命令窗口中输入.

`goal=[1,1];`

`weight=[1,1];`

```

Aeq=[1,1,1];
beq=[6];
x0=[1;1;1];
lb=[0,0,0]; %也可以写 lb=zeros(3,1);
[x,fval]=fgoalattain('myfun',x0,gcal,weight,[ ],[ ],Aeq,beq,lb,[ ])

```

③得到结果.

```

x =
    3.2727
    1.6364
    1.0909
fval =
    8.9422    19.6364

```

例 2. 某钢铁公司因生产需要欲采购一批钢材，市面上的钢材有两种规格，第 1 种规格的单价为 3500 元/t，第 2 种规格的单价为 4000 元/t. 要求购买钢材的总费用不超过 1000 万元，够得钢材总量不少于 2000t. 问如何确定最好的采购方案，使购买钢材的总费用最小且购买的总量最多.

解：设采购第 1、2 种规格的钢材数量分别为 x_1 和 x_2 . 根据题意建立如下多目标优化问题的数学模型.

$$\begin{aligned}
 \min \quad & f_1(x) = 3500x_1 + 4000x_2 \\
 \max \quad & 3500x_1 + 4000x_2 \leq 10000000 \\
 & x_1 + x_2 \geq 2000 \\
 & x_1, x_2 \geq 0
 \end{aligned}$$

①建立 M 文件. 在 Matlab 编辑窗口中输入：

```

function f=myfun(x)
f(1)= 3500*x(1)+4000*x(2);
f(2)=-x(1)-x(2);

```

②在命令窗口中输入.

```

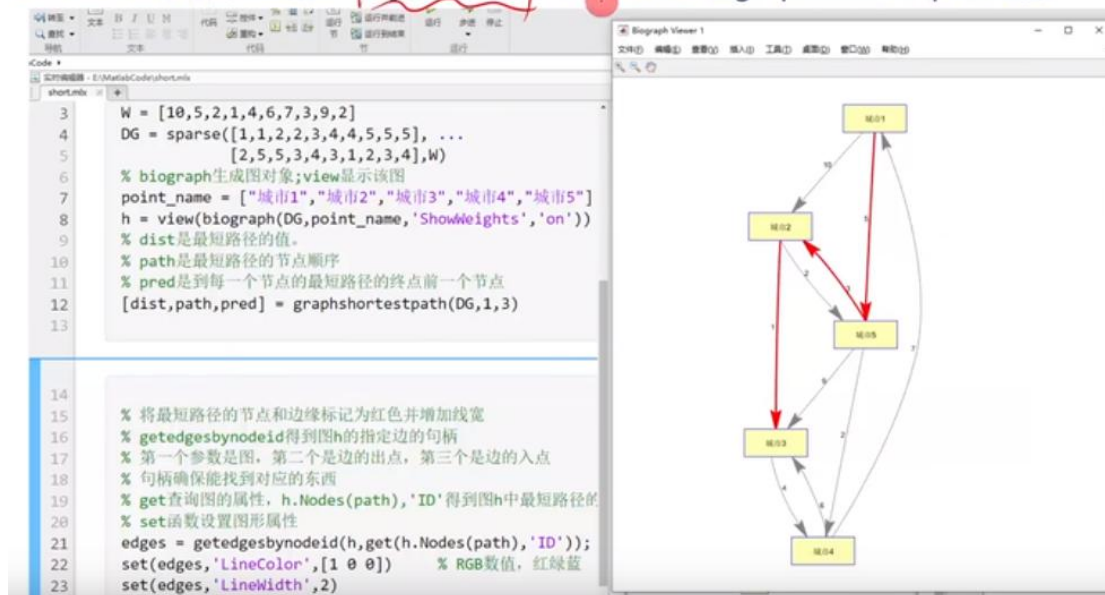
gcal=[10000000,-2000];
weight=[10000000,-2000];
x0=[1000,1000];
A=[3500,4000;-1,-1];
b=[10000000,-2000];
lb=[0,0]; %也可以写 lb=zeros(3,1);
[x,fval]=fgoalattain('myfun',x0,goal,weight,A,b,[ ],[ ],lb,[ ])

```

③得到结果.

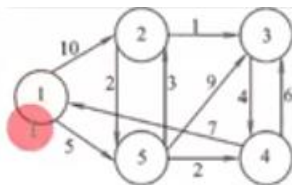
④最短路径：

□ MATLAB求解最短路径: Dijkstra算法, 或MATLAB的graphshortestpath函数



以上为完整代码。

最后两行 set 函数改颜色和宽度。改颜色时[1 0 0]表示全红，[0 1 0]全绿，[0 0 1]全蓝
前面 edges 处，Nodes(path)表示选取前面得到的几个节点，因为这几个节点属于 h，前面加上 h.，右边的 ID 是 matlab 自带，表示找到节点 ID，写代码时不用写



```

% sparse生成稀疏矩阵，也就是除了注明的几个元素外，其余都
% sparse里第一个和第二个矩阵相同位置的元素值就是非零元素的
% 非零元素的值
% W是每条边的权值
W = [10,5,2,1,4,6,7,3,9,2]
DG = sparse([1,1,2,2,3,4,4,5,5,5], ...
           [2,5,5,3,4,3,1,2,3,4],W)

```

DG:

$$\begin{bmatrix}
 0 & 10 & 0 & 0 & 5 \\
 0 & 0 & 1 & 0 & 2 \\
 0 & 0 & 0 & 4 & 0 \\
 7 & 0 & 6 & 0 & 0 \\
 0 & 3 & 9 & 2 & 0
 \end{bmatrix}$$

(部分代码)

矩阵 DG 中第一行第一列的 0 表示从 1 到 1 路径为 0，第一行第二列的 10 表示从节点 1 到节点 2 路径为 10，没有从 1 到 3 的直接路径，第一行第三列为 0……

sparse 函数能快速生成如上稀疏矩阵。第一、二个矩阵代表索引，第三个矩阵代表两个索引组成的边对应的权值。如上图：第一、二个矩阵的第一个数 1 和 2，代表从节点 1 到 2 这条边，第三个矩阵第一个数是 10，表示从 1 到 2 的路径（权值）是 10；同理，第一、二、三矩阵第二个数分别是 1，5，5，表示从节点 1

到 5 权值是 5。部分路径为 0，在三个矩阵中没有写出，最后生成的稀疏矩阵中这些都是 0。

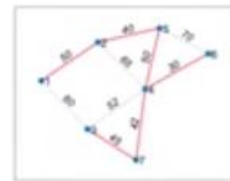
上图 sparse 函数内第一个矩阵后的“...”是第一到第二个矩阵换行后自动产生，表示它们本应该在同一行。

⑤最小生成树:

I 用 matlab 的 minspantree 函数求解

```
s = [1,1,2,2,3,3,4,4,4,5];
t = [2,3,4,5,4,7,5,6,7,6];
weights = [50,60,65,40,52,45,50,30,42,70] ;
% 生成无向图，其中s和t对应元素代表着边，weights是权值
G = graph(s,t,weights);
% 求出最小生成树，得到的T包含最小生成树的节点和对应边的权值
T = minspantree(G);

% p = plot(G)就能把图片展现出来，后面是为了美观设置字体等
p = plot(G, 'EdgeLabel', G.Edges.Weight, "MarkerSize", 8,
% 突出显示绘制的图中的节点和边
highlight(p,T, 'EdgeColor', 'red', "LineWidth", 3))
```



从图中任意顶点起步，沿着边一定可以到达所有的其他顶点，这种图叫连通图

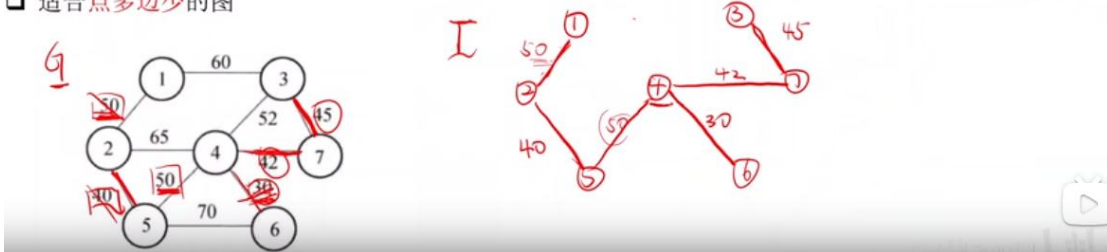
与最短路径区别：最短路径是针对某一顶点作为起点而言，最小生成树是所有顶点连通且总路径最小

代码中：前两个矩阵 s,t 表示顶点，对应位置意味着某一条边，第三个矩阵表示边的权值。如：各矩阵第三个元素分别是 2, 4, 65，代表 2 到 4 这一条边的权值是 65；因为是无向图，矩阵中已经表示 1 到 2 的情况下，不用再写 2 到 1；graph 生成图；minspantree 函数求最小生成树；后两行不必要(highlight 突出显示，边加红加粗)

□ Kruskal算法

- 1、把图G中的所有边全部去掉，得到所有单独的顶点V构成图 $T \equiv (V, \{\})$ ，其中V是顶点集合
- 2、从G中取出当前权值最小的边，如果该边加入T的边集合后T不形成回路，则加入T；否则舍弃
- 3、重复第2步，直到T中有n-1条边(n是顶点数)
- 若第2步中遇到两条权值相同的最小权值边，任选一条即可，所以最小生成树可能不唯一，但权值之和相同

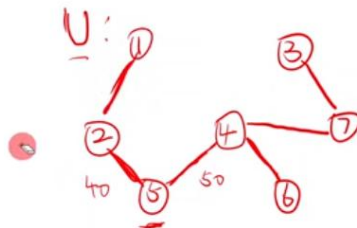
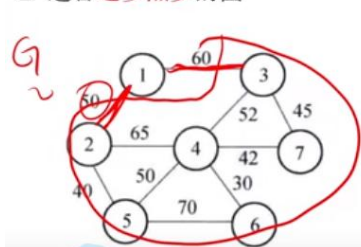
□ 适合点多边少的图



□ Prim算法

- 1、设置一个图U，将原图G中任意一顶点取出加入U中
- 2、在所有的 $u \in U, v \in (V-U)$ 的边 (u, v) 中找到一条权值最小的边，并入图U中
- 3、重复步骤2，直到U中包含了所有顶点
- 若第2步中遇到两条权值相同的最小权值边，任选一条即可，所以最小生成树可能不唯一，但权值之和相同

□ 适合边多点的图



⑥ 灰色预测模型:

数据少，看不出明显规律，适合用灰色预测模型

GM(1,1)模型：制造规律：数据累加生成序列

累加生成序列： $x^{(1)}(k) = \sum_{i=1}^k x^{(0)}(i)$

得到新序列看起来像一个指数曲线（直线）时，可用一个指数曲线乃至一条直线的表达式来逼近这个新序列。构建一阶常微分方程求解。

□ 设 $x^{(1)}$ 满足： $\frac{dx^{(1)}}{dt} + ax^{(1)} = u$ 若 a 和 u 已知则可直接求解出结果（高数基本知识）

• 要预测下一年数值，就解出微分方程；要解微分方程，就要知道 a 和 u

□ 想求 a 和 u ，就要用最小二乘法，通过最小化误差的平方和寻找数据的最佳函数匹配

□ 但已知的数据是离散的而不是连续的，所以 $\frac{dx^{(1)}}{dt}$ 写成 $\frac{\Delta x^{(1)}}{\Delta t}$ $\rightarrow \sum_{i=1}^n \lambda^{(i)} u_i \rightarrow \sum_{i=1}^n \lambda^{(i)} u_i$

• 注意！ $\Delta t = (t+1) - t = 1$ ，始终为1；而 $\Delta x^{(1)} = x^{(1)}(t) - x^{(1)}(t-1) = x^{(0)}(t)$

• 得到方程 $x^{(0)}(t) + ax^{(1)}(t) = u$

• 即 $x^{(0)}(t) = -ax^{(1)}(t) + u$ $\hat{a}, \hat{u}$

• 式子左边是已知数据，右边就是含有未知数的函数，此时就可用最小二乘法

最小二乘法即求回归方程

□ 为了(更合理)修正 $x^{(1)}(t)$ 为均值生成序列 $z^{(1)}(t)$

- 考虑原方程中有 $\frac{dx^{(1)}}{dt}$, 因此将 $x^{(1)}(t)$ 改为取前后两个时刻的均值更合理
- 对累加生成的序列 $x^{(1)}$ 再对其结果进行一次均值生成: $z^{(1)}(t) = 0.5x^{(1)}(t) + 0.5x^{(1)}(t-1)$, $t=2, \dots, n$
- 即方程改为 $x^{(0)}(t) = u - 0.9z^{(1)}(t)$, 等号左边含有7个已知数据, 等号右边的函数有两个未知参数

□ 最小二乘法求解

- 最小二乘法就是使得拟合函数求的值与已知数据的平方差最小
- 方程矩阵形式 $Y=BU$
- $$\begin{bmatrix} x^{(0)}(2) \\ x^{(0)}(3) \\ \vdots \\ x^{(0)}(N) \end{bmatrix} = \begin{bmatrix} -\frac{1}{2}[x^{(1)}(2) + x^{(1)}(1)] & 1 \\ -\frac{1}{2}[x^{(1)}(3) + x^{(1)}(2)] & 1 \\ \vdots & \vdots \\ -\frac{1}{2}[x^{(1)}(N) + x^{(1)}(N-1)] & 1 \end{bmatrix} \begin{bmatrix} a \\ u \end{bmatrix}$$

Y B U
- $$x^{(0)}(t) = \frac{-a}{0.5} z^{(1)}(t) + b$$
- 最小二乘法也就是求 $(Y - BU)^T(Y - BU)$ 取最小值时的U
- 求解U的估计值为 $\hat{U} = [\hat{a}, \hat{u}]^T = (B^T B)^{-1} B^T Y$ (最小二乘法的求解过程涉及矩阵求导等, 在此不展开讲)

□ 参数a和u已求出, 代入原微分方程, 求出 $\frac{dx^{(1)}}{dt} + ax^{(1)} = u$ 的解 (高数知识)

- $\hat{x}^{(1)}(k+1) = (x^{(0)}(1) - \frac{b}{a})e^{-ak} + \frac{b}{a}$, $k=0, 1, \dots$
- 当式中的 $k=1, 2, 3, \dots, 6$ 时求得的为拟合值, 大于等于7时为预测值
- 分别取 $k=7$ 和 6 , 得 $\hat{x}^{(1)}(8)$ 和 $\hat{x}^{(1)}(7)$; 就能预测下一年的道路噪声平均声级 $\hat{x}^{(0)}(8) = \hat{x}^{(1)}(8) - \hat{x}^{(1)}(7)$

□ 但求得的这个模型靠不靠谱? 需要进行模型检验

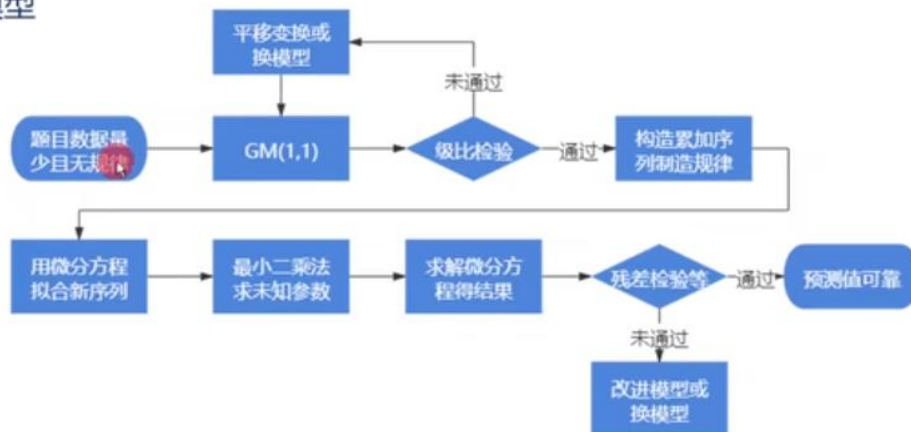
- 模型检验就是看按照 $\hat{x}^{(1)}(k+1)$ 求得的拟合值和实际值相差大不大
- 残差检验: $\varepsilon(k) = \frac{x^{(0)}(k) - \hat{x}^{(0)}(k)}{\hat{x}^{(0)}(k)}$, $k=1, 2, \dots, n$; 其中 $\hat{x}^{(0)}(1) = x^{(0)}(1)$
- 如果残差 $\varepsilon(k) < 0.2$, 好! 如果 $\varepsilon(k) < 0.1$, 很好!
- 如果大于0.2怎么办? 换模型吧....灰色预测不适合本题. (此外还有级比偏差检验等, 不再介绍)

□ 在建模最开始, 需要进行数据的级比检验

- 为了确定数据使用GM(1,1)模型的可行性, 避免白忙活, 需要数据的级比检验:
- 计算 $\lambda(k) = \frac{x^{(0)}(k-1)}{x^{(0)}(k)}$, $k=2, 3, \dots, n$. 如果 $\lambda(k)$ 在区间 $(\frac{2}{e^{n+1}}, \frac{2}{e^{n+2}})$ 内, 说明可用GM(1,1)模型;
- 如果在区间外, 可尝试平移变换, 也就是给每个数据都加上任意常数c, 看是否在区间内; 求解后再减去

□ GM(1,1)模型

□ 整体流程



□ 优点: 数据少且无明显规律时可用, 利用微分方程挖掘数据本质规律

□ 缺点: 灰色预测只适合短期预测、指数增长的预测

GM(1,1)模型代码求解:

原始数据级比检验

计算 $\lambda(k) = \frac{x^{(0)}(k-1)}{x^{(0)}(k)}$ ，如果所有 $\lambda(k)$ 在区间 $\left(e^{-\frac{2}{n+1}}, e^{\frac{2}{n+1}}\right)$ 内，说明可

用GM(1,1)模型

```
clc,clear
x0=[71.1 72.4 72.4 72.1 71.4 72.0 71.6]'; % 注意这里为列向量
n=length(x0);
lamda=x0(1:n-1)./x0(2:n); % 计算级比
range=minmax(lamda'); % 计算级比的范围
lamrange_min=exp(-2/(n+1)); % 允许的范围下界
lamrange_max=exp(2/(n+1)); % 允许的范围上界
% 原始数据的级比检验
if (range(1)>lamrange_min && range(2)<lamrange_max)
    fprintf("原始数据通过级比检验")
else
    fprintf("原始数据未能通过级比检验!!!")
    return
end
```

length()计算 x0 内元素数量；计算级比即求对应元素的比，1 对应 2，n-1 对应 n

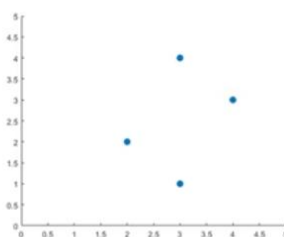
⑦TOPSIS 法：

□TOPSIS法（理想解法）

□ 简单示例

- 法外狂徒张三所在的团伙要评选盗圣，每个人的“事迹”如表所示
- 试对这一伙人进行评估，评选出盗圣

姓名	盗窃成功次数	盗窃总价值（万）
张三	3	4
李四	4	3
王五	2	2
赵六	3	1



□ TOPSIS法基本概念

- 总共4人，有4种可能的结果/方案，方案集： $D = \{d_1, d_2, d_3, d_4\} = \{\text{张三}, \text{李四}, \text{王五}, \text{赵六}\}$
- 方案 d_i 的属性值（评价指标）构成向量 $[a_{i1}, a_{i2}]$ ，代表空间中的一个点
- 例如 $[a_{21}, a_{22}] = [4, 3]$ 代表二维空间中(4,3)这个点，现实意义是李四的盗窃成功次数和总价值

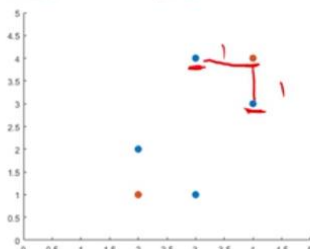
例如：已知师生比、科研经费、逾期毕业率等，评估学生教育实际情况

该方法适用于客观评估类型题目，题目要提供了足够的评价指标和数据，同时评价指标类型差异较大（数值、比值、百分比，有正面指标也有负面指标）。充分利用原始数据，无需数据检验，在评价指标过多时避免过于复杂。

□ 正理想解和负理想解

- 理想中的盗圣：成功次数4，总价值4（集合了两条指标的最优值），称为正理想解
- 理想中的失败者：成功次数2，总价值1（集合了两条指标的最差值），称为负理想解
- 在一个二维坐标系中，以两个评价指标作为x和y，画出每个点
- 4人中距离理想中的盗圣最近，距离理想中的失败者最远的人，为盗圣

姓名	盗窃成功次数	盗窃总价值 (元)
张三	3	40000
李四	4	30000
王五	2	2
赵六	3	1
正理想解	4	4
负理想解	2	1



- 思考：如果评价指标是3个呢？4个呢？n个呢？如果评价指标不在一个数量级呢？如果评价指标有负面的（例如被逮捕次数）呢？

在二维空间内分别画出正负理想解所代表的点，再依次画出4人所代表的点，求他们代表的点与正负理想点的距离。距离正理想解的点同样近时，看谁距离负理想解最远。

评价指标有负面时，正理想解取其最小值而不取最大值

因为数据不是越大越好，且不同指标的单位、数量级不同，要进行数据预处理：

□ 数据预处理

□ 指标数据并不都是越大越好

- 效益型属性：越大越好的属性，例如人均专著、科研经费
- 成本型属性：越小越好的属性，例如逾期毕业率
- 区间型属性：在某一区间内最好，过大或过小都不好，例如生师比

代码求解：

□ 求解过程

- 输入原始数据
- 数据预处理：区间型属性的变换
- 数据预处理：向量规范化
- 加权处理
- 按照公式求正负理想解、各方案与正理想解的距离
- 求各方案的综合评价指数

以此题为例：

□ 评价研究生教育

- 现有5所研究生院的数据，如表所示
- 请建立模型，客观地评价研究生教育的实际状况和各研究生院的教学质量

院校编号	人均专著 (本/人)	生师比	科研经费 (万元/年)	逾期毕业率 (%)
1	0.1	5	5000	4.7
2	0.2	6	6000	5.6
3	0.4	7	7000	6.7
4	0.9	10	10000	2.3
5	1.2	2	400	1.8

```
% 输入原始数据
a=[0.1 5 5000 4.7
0.2 6 6000 5.6
0.4 7 7000 6.7
0.9 10 10000 2.3
1.2 2 400 1.8];
[m,n]=size(a); % m是行数（方案数），n是列数（属性数）
```

区间型属性的变换

设第 j 个指标的最优属性区间为 $[a_j^0, a_j^*]$, a_j' 为无法容忍下限, a_j'' 为无法容忍上限, 则:

$$b_{ij} = \begin{cases} 1 - (a_j^0 - a_{ij}) / (a_j^0 - a_j'), & a_j' \leq a_{ij} < a_j^0 \\ 1, & a_j^0 \leq a_{ij} \leq a_j^* \\ 1 - (a_{ij} - a_j^*) / (a_j'' - a_j^*), & a_j^* < a_{ij} \leq a_j'' \\ 0, & \text{else} \end{cases}$$

```
% 定义函数句柄b2, 相当于在此定义了一个函数
% @(qujian,lb,ub,x)意味着调用b2时, 需要给它括号里的四个参数
% (x>=lb & x<qujian(1))是逻辑与, 如果&符号左右两边的条件都满足,
% 四种情况是互斥的, 所以可以乘以各自区间条件逻辑计算后相加
a2=@(qujian,lb,ub,x)...
(1-(qujian(1)-x)./(qujian(1)-lb)).*(x>=lb & x<qujian(1))...
+(x>=qujian(1) & x<=qujian(2))+...
(1-(x-qujian(2))./(ub-qujian(2))).*(x>qujian(2) & x<=ub);

% 设最优区间等参数
qujian=[5,6]; lb=2; ub=12;
a(:,2)=a2(qujian,lb,ub,a(:,2)); % 对生师比进行变换
```

⑧微分方程传染病预测模型:

□从最简单的指数传播模型说起

□ 传染病预测问题

- 不同类型传染病的发病机理和传播途径各有特点
- 有的传染病, 在得过一次后可获得免疫力, 但有的则不会
- 有的传染病具有潜伏期, 有的则没有
- 需要对不同类型的传染病建立相应合适的预测模型

□ 从最简单的看起: **指数传播模型**

1. 假设所研究的区域是封闭区域, 在一定时期内人口总量不变, 不考虑迁入和迁出
2. 在 t 时刻患病人数 $N(t)$ 是随时间连续变化的、可微的函数
3. 每个病人在单位时间内会传染到的人数为大于0的常数 λ

□ 指数传播模型

□ 模型的建立

- 设 $N(t)$ 为 t 时刻患病人数, 则 $t + \Delta t$ 时刻的患病人数为 $N(t + \Delta t)$
- 则从 $t \rightarrow t + \Delta t$ 时间内, 净增的患病人数为 $N(t + \Delta t) - N(t)$
- 根据假设3 (每个病人在单位时间内会传染到的人数为大于0的常数 λ), 有:

$$N(t + \Delta t) - N(t) = \lambda N(t) \Delta t$$

- 注意, λ 在模型中始终是常数 (每个病人在单位时间内会传染到的人数)

□ 微分方程

- 基于上一页的第2条模型假设, 在上面公式等号两边同时除以 Δt , 并令 $\Delta t \rightarrow 0$
- 可得微分方程:

$$\begin{cases} \frac{dN(t)}{dt} = \lambda N(t), & t > 0 \\ N(0) = N_0 \end{cases}$$

- 可求得该模型的解析解: $N(t) = N_0 e^{\lambda t}$

指数传播模型一般适应于传染病爆发初期。封闭区域内人数有限, 患病人数越来越多时, 没有多少人可供感染, 新增患病人数的导数也会减少, 这时可改进使用 SI 模型

□ SI 模型

□ 模型假设

1. **人口总数**: 所研究的区域内人口总数为常数 N , 既不考虑生死, 也不考虑迁移
2. **两类人群**: 人群分为易感染者 (susceptible) 和已感染者 (infective), 设 t 时刻两类人群在总人口中所占的比例分别为 $s(t)$ 和 $i(t)$, 显然 $s(t) + i(t) = 1$
3. **日感染率**: 每个病人在单位时间 (每天) 内接触的平均人数为常数 λ , 称为日感染率; 当病人所接触的是健康者时, 会将其感染成病人
4. **不考虑治愈**: 每个病人得病后在传染期内无法治愈, 且不会死亡

□ 注意事项

- 现实中, 地区人数并不会真的为常数, 总有出生率、死亡率、迁入和迁出等
- 但如果把这些因素考虑进模型, 模型会非常复杂; 而本题重心是传染病
- 再次强调模型假设的目的: 简化问题

□ 模型建立

- **细节**: 第3条假设中 λ 是1个病人单位时间接触的平均人数, 接触的人中既有病人也有健康者
- 则1个病人单位时间内可使 $\lambda s(t)$ 个健康者变为病人
- 在 t 时刻病人总数 $Ni(t)$, Δt 时间内会新增 $\lambda s(t) Ni(t) \Delta t$ 个病人, 则单位时间内新增病人数:

$$\frac{Ni(t + \Delta t) - Ni(t)}{\Delta t} = \lambda N s(t) i(t)$$

- 令 $\Delta t \rightarrow 0$, 得微分方程

$$\frac{di(t)}{dt} = \lambda s(t) i(t)$$

- 根据第2条假设, 由于 $s(t) + i(t) = 1$, 所以可写作

$$\frac{di(t)}{dt} = \lambda i(t) (1 - i(t))$$

设 $t=0$ 时, 患病人数占总人口比例 $i(0)=i_0$, 可求得微分方程的解
比赛时需要根据数学结果, 分析求解结果的现实意义, 写进论文

SI 模型未考虑病人得病后可以治愈，要进一步改进建立 SIS 模型

□ SIS模型

- SIS模型在SI模型假设的基础上，进一步假设：

1、治愈比例：每天被治愈的病人人数占病人总数的比例为 μ

2、无免疫性：病人被治愈后成为仍可被感染的健康者

- 得到SIS模型：

$$\begin{cases} \frac{di(t)}{dt} = \lambda i(t)(1 - i(t)) - \mu i(t), & t > 0, \\ i(0) = i_0. \end{cases}$$

- 可见，SIS模型是比原SI模型多了一项“ $-\mu i(t)$ ”

□ SIS模型解析解

- 该模型的解析解为：

dsolve

$$i(t) = \begin{cases} \left[\frac{\lambda}{\lambda - \mu} + \left(\frac{1}{i_0} - \frac{\lambda}{\lambda - \mu} \right) e^{-(\lambda - \mu)t} \right]^{-1}, & \lambda \neq \mu \\ \left(\lambda t + \frac{1}{i_0} \right)^{-1}, & \lambda = \mu \end{cases}$$

- 令 $\sigma = \frac{\lambda}{\mu}$ 为传染强度（病人日接触率/病人每日被治愈比例）
- 带入上一页模型的公式，得微分方程和解析解：

$$\begin{cases} \frac{di}{dt} = -\lambda i \left[i - \left(1 - \frac{1}{\sigma} \right) \right], & t > 0 \\ i(0) = i_0 \end{cases}$$

$$i(t) = \begin{cases} \left[\frac{1}{1 - \frac{1}{\sigma}} + \left(\frac{1}{i_0} - \frac{1}{1 - \frac{1}{\sigma}} \right) e^{-\lambda \left(1 - \frac{1}{\sigma} \right) t} \right]^{-1}, & \sigma \neq 1 \\ \left(\lambda t + \frac{1}{i_0} \right)^{-1}, & \sigma = 1 \end{cases}$$

□ 显然，传染强度 $\sigma = \frac{\lambda}{\mu}$ 为一个阈值

- 若 $\sigma \leq 1$ ，现实意义“病人日接触率 $\leq$ 病人每日被治愈比例”
- 即新增病人少于被治愈病人，那么随着 $t \rightarrow \infty$ ，终所有病人都会被治愈
- 若 $\sigma > 1$ ，现实意义“病人日接触率 $>$ 病人每日被治愈比例”
- 即新增病人多于被治愈病人，那么随着 $t \rightarrow \infty$ ，总有一定比例的人口会被传染成病人

□ SIR模型

□ 模型的建立

- 根据以上分析，建立微分方程传染病预测SIR模型：

$$\begin{cases} \frac{di}{dt} = \lambda s(t)i(t) - \mu i(t) \\ \frac{ds}{dt} = -\lambda s(t)i(t) \\ \frac{dr}{dt} = \mu i(t) \\ i(0) = i_0, s(0) = s_0, r(0) = 0 \end{cases}$$

□ 模型分析

- SIR模型形式是多个相互关联的系统变量之间的常微分方程组，属于典型的系统动力学模型
- 更复杂的情况，考虑有些传染病具有潜伏期，考虑一类人为潜伏者，建立SEIR模型
- 类似的问题：河流各类污染物质的耗氧、复氧、吸附、沉降等
- 该类问题往往难以求得精确的解析解，可以使用MATLAB求数值解

代码求解 SIR 模型微分方程：

微分方程SIR模型求解

函数ode45详细的官方说明：

[https://ww2.mathworks.cn/help/matlab/ref/ode45.html?](https://ww2.mathworks.cn/help/matlab/ref/ode45.html?s_tid=srchtitle_ode45_1#d123e934673)

[s_tid=srchtitle_ode45_1#d123e934673](https://ww2.mathworks.cn/help/matlab/ref/ode45.html?s_tid=srchtitle_ode45_1#d123e934673)

```
clc,clear
% 参数初始化
lam = 0.4;      % 日接触率
mu = 0.1;       % 日治愈率
I = 0.4;
S = 0.5;

% ode45是一个通用型ODE求解器
tspan = [0:1:50]; % 变量t范围0到50
y0 = [I S];       % 传递模型参数
```

⑨马尔科夫预测：

□ 时齐性

□ 时齐的马尔科夫链

- 若第 n 天张三处于状态 i ，则经过了 m 天后，张三处于状态 j 的概率满足：

$$P\{X_{n+m}=j | X_n=i\} = P_{ij}(m)$$

- 则称 $\{X_n, n=1,2,\dots\}$ 为时齐的马尔科夫链

例如已知第3天为状态1（行窃），求7天后（第10天）处于状态4（蹲大牢）的概率：

- $P\{X_{10}=4 | X_3=1\} = P_{14}(7)$
- 或者已知第100天为状态1（行窃），求7天后（第107天）处于状态4（蹲大牢）的概率：
- $P\{X_{107}=4 | X_{100}=1\} = P_{14}(7)$

⑩模拟退火算法：

当前处于局部最优解，随机试探新解，有更好的就直接选择新解，没有更好的则以一定概率选择新解，继续随机试探

□ 贪心算法失效——陷入局部最优

□ 从爬山说起

- 假如北海想要在日落前爬上一座山的最高峰（求最优解）
- 但山中云雾缭绕、视野受限，只能看到当前位置附近一定范围内的情况
- 贪心算法：在视野内选取一个点，如果更高，就过去；否则就不去
- 直到视野内没有更高的点，则输出当前解作为最终解（视频中有板书过程）
- 显然，当北海走到某个小山峰时，四周看不到更高的点，就不会走下山去、寻找更高的峰
- 从而陷入了局部最优解，课程试看部分

模拟退火思想：发现有一点更低，有一定概率走下山峰并走向新的更高峰，寻找全局最优

□ 算法关键点

- 时间有限，需设置终止条件（“退火”过程中温度不断降低，剩余时间不多时就别乱跑）
- 且算法需要做到：若当前所处的山峰越高，前往低点的概率越低（概率的表达式）
- 这样才能使求得最优解的概率更大
- 关键在于，“概率”怎么求

□ 适用赛题

□ 可行解过多、NP-hard问题

□ NP-hard问题

- 旅行推销员问题 (Travelling salesman problem, TSP)：给定一系列城市和每对城市之间的距离，求解访问每一座城市一次并回到起始城市的最短回路
- TSP问题是组合优化中的一个NP-hard问题，该问题的可行解是所有顶点的全排列，随着顶点数的增加，会产生组合爆炸，传统算法难以求解
- 此时就适合用模拟退火等启发式算法求近似解

□ 适用赛题特点

- 有些规划类问题的可行解过多，传统算法运算时间过长
- 能够列出可行解的任一排列（例如题目要求最短回路，起码能先写出个回路作为初始化）

□ 国赛题目：高等教育学费标准

- 假设高校所需1000万经费，涉及多种筹款路径，理论上的筹款组合方案过多，若暴力求解可能费时几十小时，此时就适合模拟退火求解

□ 典型例题

□ 遍历全国省会

- 已知全国34个省会城市（包括直辖市、自治区首府和港澳台）的经纬度坐标（第一个为北京）
- 现在需要从北京出发到所有城市视察
- 要求每个城市只能到达一次，并最终回到北京
- 求视察路线方案，使得总路径最短

□ 问题分析

- 若把所有可能的路线方案都列出来再求最小值，则有33!（33的阶乘）种方案
- 这是10³⁶数量级，运算时间会很长
- 此时可考虑使用模拟退火

- 本期课程的原理讲解内容对于部分同学来说较难理解，应结合代码理解

- 本题的解空间就是固定起点和终点的所有可行路线的集合
- 题目要求以北京作为起点和终点，那么设起点北京为1号城市x₁
- 余下需要遍历的33个城市依次设为2, 3, ..., 34号城市x₃₄
- 最后还需回到北京，设终点北京为35号城市x₃₅
- 因为起点与终点固定，则共有33!个可行解

终点设为x₃₅。若仍为x₁，可看作一座城市被访问两次，写代码会有不便。

□ 目标函数：总路程最短

- 从1号x₁出发遍历34个城市最后回到1号x₃₅
- 第i号城市x_i与第i+1号城市x_{i+1}之间的路径长度设为d_{x_ix_{i+1}}
- 则目标函数：

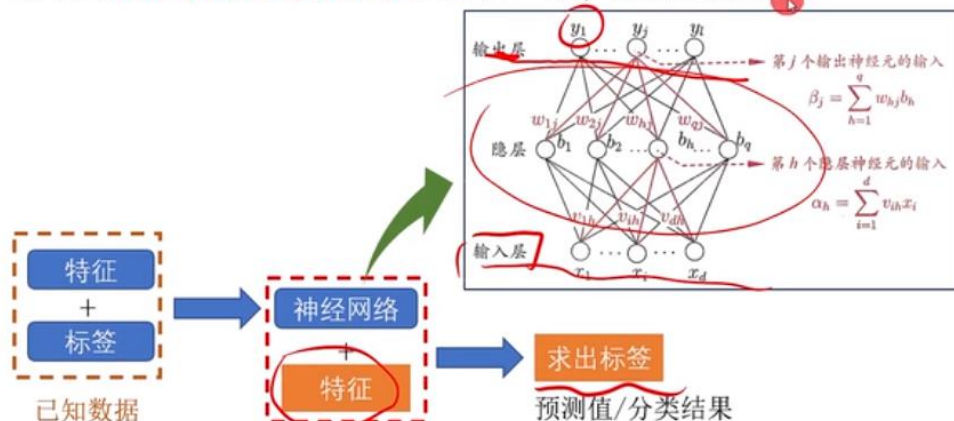
$$\min f(x_1, x_2, \dots, x_{35}) = \sum_{i=1}^{34} d_{x_i x_{i+1}}$$

x₁ 和 x₃₅ 固定，剩下元素随机排列

11. 神经网络:

翻译翻译什么叫做“神经网络”

- 神经网络由多层的神经元(节点)构成, 不同层之间的神经元通过权重连接
- 隐藏层和输出层的神经元中有激活函数, 收到的数据作为自变量, 计算出结果
- 计算出的结果与权重相乘后累加传递到下一层, 或输出最终结果



适用赛题

预测类问题

- 预测未来的结果, 一般为回归问题, 即输出值 y 是连续的
- 美赛2022年C题比特币和黄金投资策略, LSTM(长短期记忆网络)进行预测
- 短时交通流量预测(小波神经网络的时间序列)
- 电力负荷预测(Elman神经网络, 具有延时记忆, 适合动态建模)

分类问题

- 对现有的对象进行分类, 一般为离散问题, 即输出值 y 是有限个离散值
- 美赛2021年C题黄蜂传播规律和目击准确性, 卷积神经网络进行图像识别
- 人体肿瘤诊断(LVQ神经网络, 无需数据预处理, 适合模式识别和优化)
- 柴油机故障判断(SOM神经网络, 适合需要聚类的问题)

评价类问题

- 嘉陵江水质评价(模糊神经网络), 高校科研能力评价(离散Hopfield神经网络)
- 本质上与预测和分类相同, 只不过题目问的是“评价结果”而已

先学会最基础的神经网络原理, 遇到题目后再去翻书现学现用

参考《MATLAB

从感知机到多层神经网络

2、如何根据“综合考虑”的结果进行判断?

- 当设线性求和的结果足够大, 与阈值的差值作为激活函数的自变量

$$\text{阶跃函数: } f(t) = \begin{cases} 1, & t \geq 0 \\ 0, & t < 0 \end{cases}$$

极端例子:

- 假设某人身高10光年、体重100吨
- 代入 $t = \sum w_i x_i - b$, 会是个非常大的数 ($t \geq 0$)
- 很容易判断已超过了8周岁 ($f(t) = 1$)



实际中需要不断修改权重 w_i 的值使模型求出预测值更接近真实值

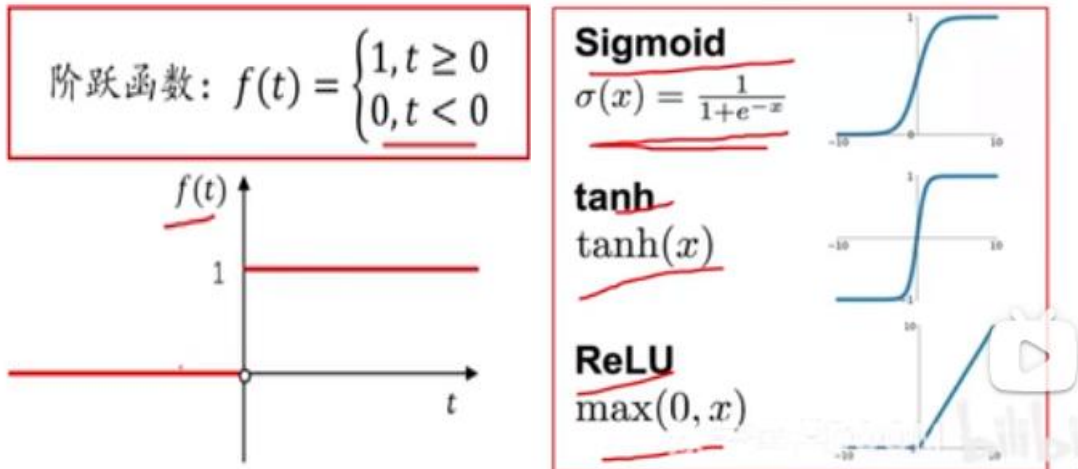
□ 优化方法: w_i 改为 $w_i + \Delta w_i = w_i + \eta(y - \hat{y})x_i$

- 当 $(y - \hat{y})$ 大于某定值时进行优化
- $\eta \in (0, 1)$ 为学习率, η 过大容易震荡, 过小则收敛速度太慢
- 达到终止条件后当前的 w_i 就作为最终的权重, 模型就建好了

基本模型: $y = f(\sum_1^2 w_i x_i - b) = f(\sum_1^3 w_i x_i) = f(t) = \begin{cases} 1, t \geq 0, \text{ 达到了8周岁} \\ 0, t < 0, \text{ 未达到8周岁} \end{cases}$

- 本示例以 $(y - \hat{y})$ 作为对模型好坏的评估指标, 更常见的有均方误差、平方绝对误差等

其他激活函数:



□ 从感知机到多层神经网络

□ 更复杂的情况

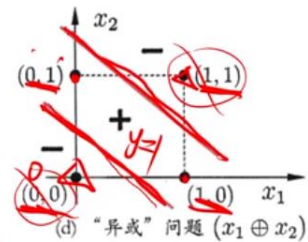
- 非线性可分问题: 异或问题, 感知机无法收敛?
- 例如: 随机选取一组两个人, 判断是否有能力生育出一个孩子
- 男性为1, 女性为0作为输入, 能生育为1, 不能生育为0作为输出

□ 使用数据集进行训练时

- 假如数据集里某一段全是 $(0, 0)$ 、 $(1, 0)$ 和 $(0, 1)$
- w_1 和 w_2 会收敛于某组定值(对应图中下方红线, 划分两种模式)

- 假如数据集里某一段全是 $(1, 1)$ 、 $(1, 0)$ 和 $(0, 1)$
- w_1 和 w_2 会收敛于另一组定值(对应图中上方红线)

但全面的数据集里肯定四种组合都有, 则权重会不断震荡、无法收敛
本质就是权重无法同时在两条边界(图中红线)收敛



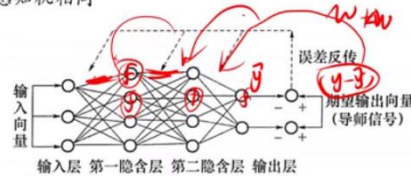
□ 从感知机到多层神经网络

□ BP神经网络 (back propagation, 误差逆传播)

- BP神经网络等于叠加多个感知机来实现非线性可分。
- 梯度下降策略: 以目标的负梯度方向对参数进行调整 (此外还有牛顿法、最小二乘等策略)
- 第 k 个训练例的均方误差:

$$E_k = \frac{1}{n} \sum_{j=1}^n (y_j^k - \hat{y}_j^k)^2$$

- 更新权重 $w_{hj} \leftarrow w_{hj} + \Delta w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}}$ 偏导展开最终表达式还是用 $(y_j^k - \hat{y}_j^k)$ 来计算
- 因此基本原理与前面感知机相同



□ 总结

- MATLAB和GitHub上都有现成的工具包, 无需原创代码
- 再复杂的神经网络, 思路都可总结为以下三点:

- 搜集数据集 (x_i, y) , 将 x_i 代入初始的神经网络模型, 计算估计值 $\hat{y}$
- 根据 $\hat{y}$ 与 y 的差异不断更改权重 w_i , 使其误差尽可能的小 (梯度下降)
- 大量数据训练后, 再输入新的数据 x , 模型就可以求出 y 用于分类/预测/评价了

- 数据集是题目给的, 或者自己搜集的, 决定了整个模型的生死
- 记得做好数据预处理 (处理缺失值、异常值)

可能出现的问题: 过拟合, 欠拟合

□ 过拟合的原因

1. 数据集有噪声 (缺失值、异常值, 无用特征等等)
2. 训练集的数据量不足 (训练集的分布与总体样本的不一致)
3. 训练模型过度, 导致模型非常复杂 (完全拟合了训练集、无法接受新数据)

□ 解决方法

- 尽量寻找更多的数据集, 并做好预处理
- 去除无用的特征 (判断一个人是否达到8岁, 与他鞋子的颜色无关)
- 早停策略: 在即将过拟合的时候立刻停止训练
- 正则化策略: 损失函数中加入惩罚项来降低模型复杂度, 特征越多惩罚越大

□ 欠拟合

- 与过拟合相反, 是由于训练不足造成的, 可在模型中增加特征来增大网络复杂度

MATLAB 神经网络工具箱: P48

12. 粒子群算法:

□ 适用赛题

□ 多目标优化问题

- 充电站布局优化 (美赛)、系统系统的设计 (国赛)、背包问题
- 当目标函数复杂、变量多 (维度高), 传统求解方法速度很慢
- 粒子群算法的最大优点: 快! (尤其是高维度且目标函数复杂的优化问题)

□ 寻优问题

- 复杂函数寻优、神经网络训练、模式分类、模糊系统控制、TSP问题等
- 直接求解, 或为这些问题的模型提供初始值或参数优化

□ 算法优缺点

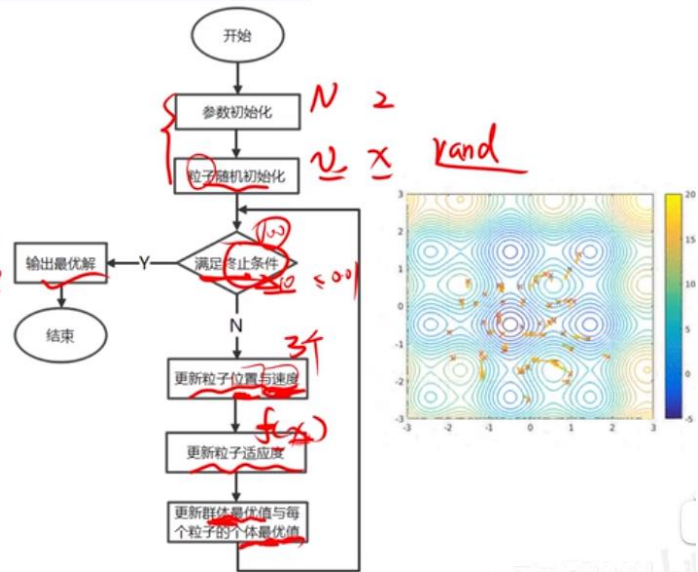
- 优点: 收敛快, 简单易实现, 几乎任何寻优问题都能用其求解
- 缺点: 局部搜索能力较差, 搜索精度不够高 (如何改进?)

□ 算法思路

□ 概念与流程图

鸟群觅食	粒子群算法
森林	解空间
鸟的当前位置	可行解 (x_1, x_2)
食物量	函数值 (适应度)
食物最多的位置	最优解
更新鸟群位置	更新可行解

微信公众号：数学建模BOOM



不要轻易使用启发式算法，如：粒子群算法，模拟退火，神经网络

13. 遗传算法

□ 适用赛题

@数学建

□ 函数优化、组合优化问题

- 对于一些非线性、多模型、多目标的函数优化问题
- 不依赖于问题的背景领域，使用方便，连续/离散、单峰/多峰等等各种形式均可

□ NP-Hard问题

- 模拟退火算法中讲过的TSP问题；背包问题、图形划分问题等
- 在NP-Hard问题方面普遍来说各类启发式算法均可

□ 优缺点

- 相比模拟退火，相比良好的全局搜索能力，不易陷入局部最优
- 相比粒子群算法，常规的遗传算法可直接求解离散问题
- 缺点：由于变异是随机的，局部搜索能力差；相对其他算法更耗时、思路复杂抽象；
- 可多种算法结合改进，例如遗传算法优化神经网络等混合算法（新手慎用）

找到不同元素好的地方，进行交叉，适当做出变异